

No matter how much activity your cluster has done, and what size the resulting pgBadger files are, once you have a report, you can point your web browser to the local files (or serve the result via a web server). You will see the report shown below. The report provides a glance at the cluster activity, including the number of statements, the time spent serving those statements, and graphs showing the statement traffic with regard to the period of time:

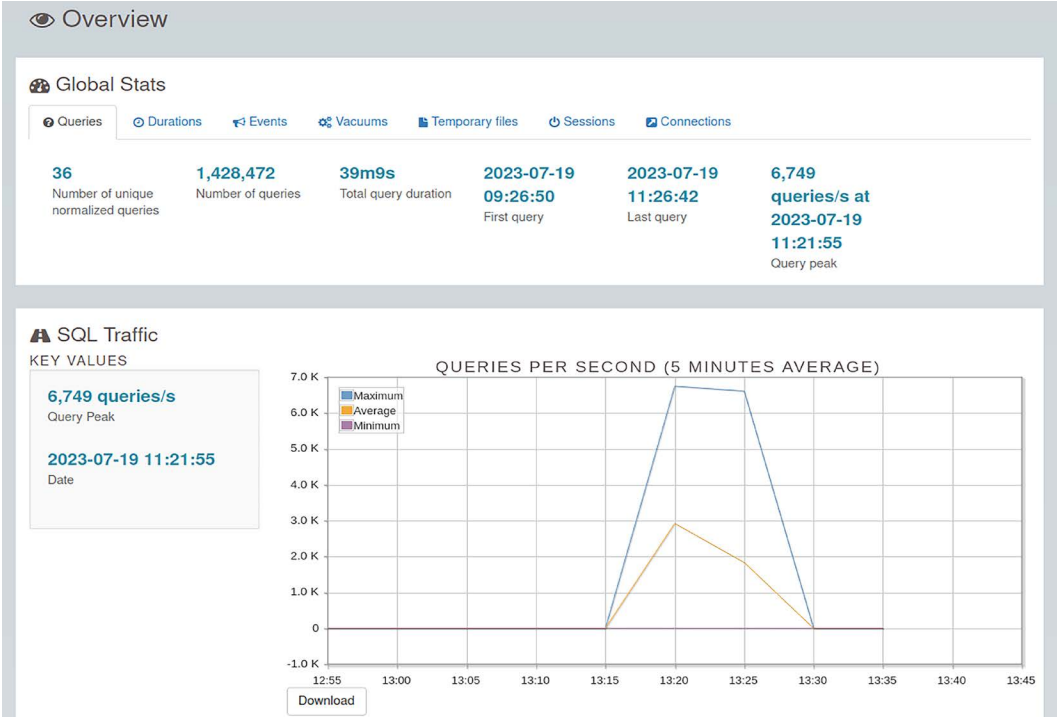


Figure 14.1: Initial page of the pgBadger dashboard

If you are using the Docker image for this chapter, you can point your web browser to the URL `http://localhost:8080/first_report.html` and you will be able to see the report.

At the top of the web page, there is a menu bar that includes several menus that allow you to look at different graphs and dashboards.

For example, the **Connections** menu allows you to get information about how many concurrent connections you had, as shown in the example here:

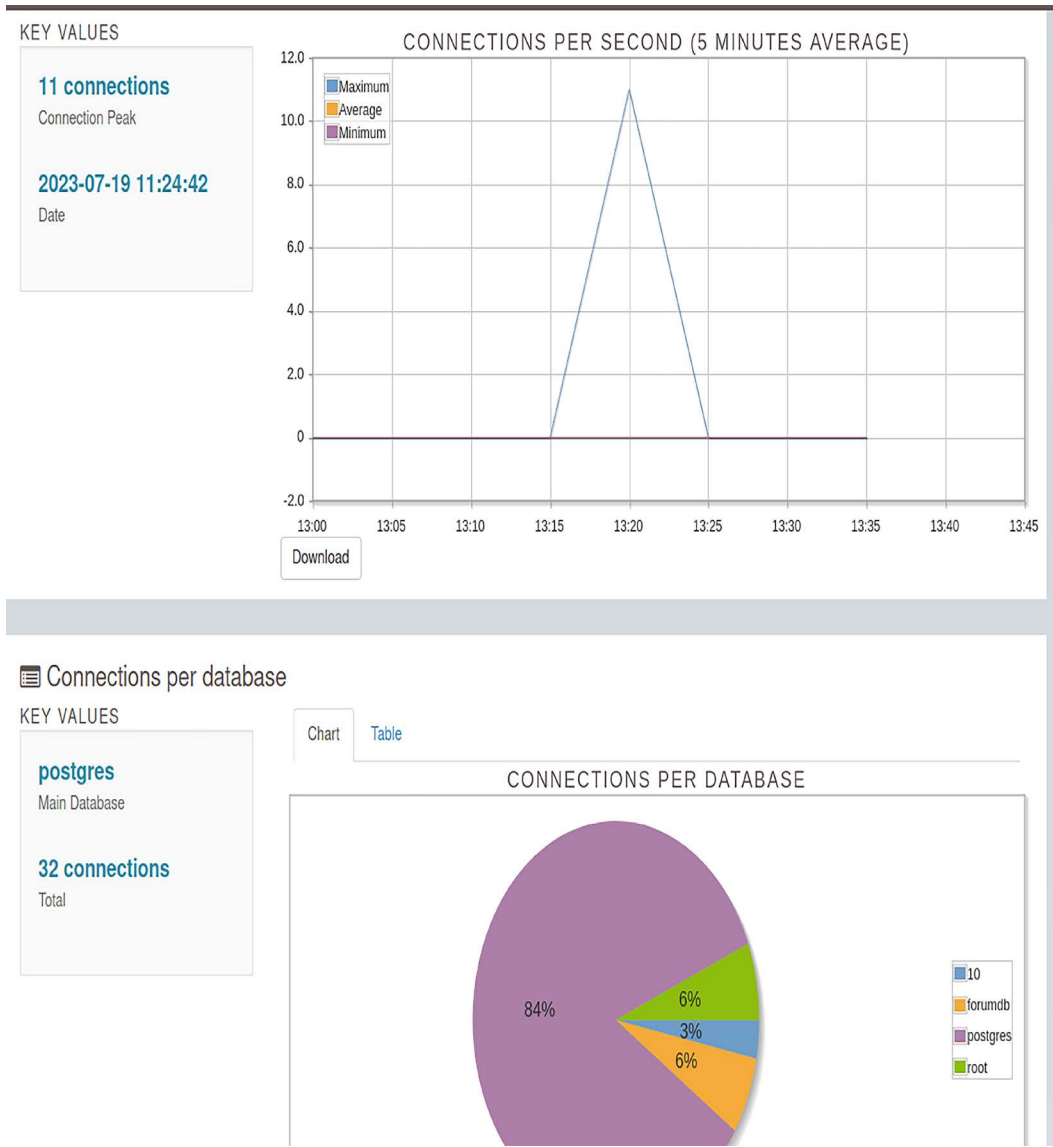


Figure 14.2: Example of the Connections dashboard

The **Queries** menu allows you to get an overview of the type and frequency of statements, as shown in the following screenshot, where the main percentage of queries was of the type **SELECT**:

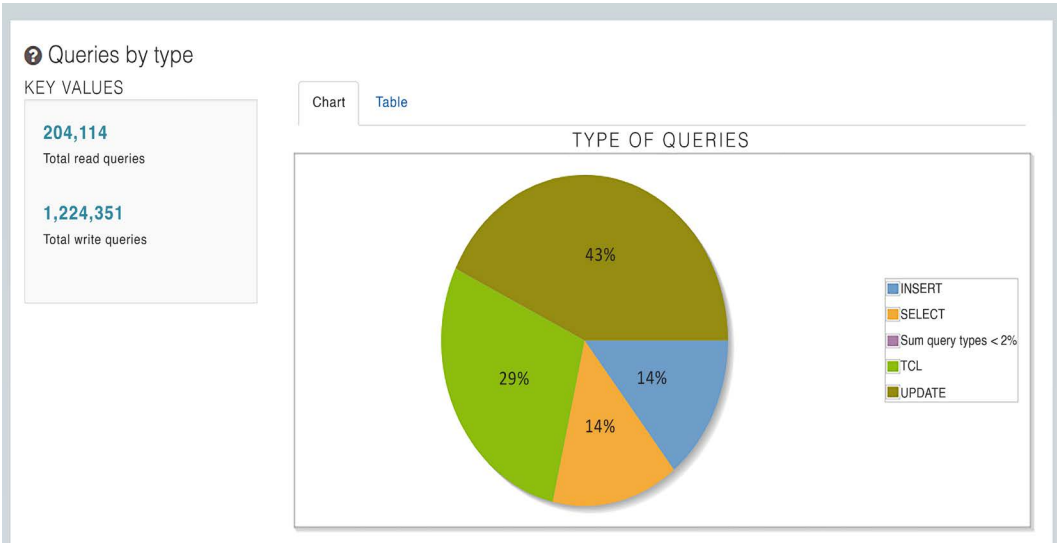


Figure 14.3: Example of the Queries dashboard

The **Top** menu allows us to see the “top events,” such as the slowest queries and the most time-consuming queries, shown respectively in the following screenshot:

Rank	Duration	Query
1	3s199ms	 SELECT count(*) FROM posts; [Date: 2020-04-17 17:23:42 - Database: forumdb - User: luca - Remote: 127.0.0.1 - Application: mysql]

Figure 14.4: An extraction of the Top Queries dashboard

pgBadger also shows a more detailed version on the same page, as shown in the following figure:

Rank	Times executed	Total duration	Min duration	Max duration	Avg duration	Query
1	294 Details	4m56s	949ms	3s199ms	1s7ms	 <code>SELECT count(*) FROM posts;</code> Examples User(s) involved App(s) involved
2	294 Details	169ms	0ms	5ms	0ms	 <code>SELECT count(*) FROM tags;</code> Examples User(s) involved App(s) involved
3	73 Details	83ms	0ms	3ms	1ms	 <code>ROLLBACK;</code> Examples User(s) involved App(s) involved
4	73 Details	2ms	0ms	0ms	0ms	 <code>BEGIN;</code> Examples User(s) involved App(s) involved
5	67 Details	357ms	3ms	16ms	5ms	 <code>TRUNCATE posts CASCADE;</code> Examples User(s) involved App(s) involved

Figure 14.5: Details about top time-consuming queries

Discussing all the features and dashboards of pgBadger is out of the scope of this book, but please see the official documentation for more details and a clear and accurate explanation of every single option.

Scheduling pgBadger

pgBadger can be used in a scheduled way so that it can continuously produce accurate reports over a specified period of time. This is possible because pgBadger includes an incremental feature, using which the report is not overwritten every time; instead, the program can produce a per-hour report and a per-week summary report.

This is handy because you can schedule pgbadger execution with, for example, cron(1) and forget about it. Let's first see how pgBadger can be run in incremental mode:

```
$ pgbadger -I --outdir /data/html -f stderr /postgres/16/data/log/
postgresql-2023-*.log
[=====>] Parsed 22008130 bytes of 22008130 (100.00%),
queries: 120569, events: 1
LOG: Ok, generating HTML daily report into /data/html/2023/07/19/...
LOG: Ok, generating HTML weekly report into /data/html/2023/week-30/...
LOG: Ok, generating global index to access incremental reports...
```

The `-I` argument specifies incremental mode, so pgBadger will produce separate files for the hourly and weekly reports. Please note that instead of specifying the output file, the `--outdir` option has been used to specify the directory to place the files in. The `-f` option tells pgBadger which kind of logs it is managing; in this example, normal text files. Lastly, as usual, there is the log file to analyze, expressed as a shell glob (`postgresql-2023-*.log`).

The end result, as you can guess from the output of the program, is that a directory tree somewhat like the following is produced:

```
$ ls -R /data/html/
/data/html:
2023  LAST_PARSED  index.html

/data/html/2023:
07  week-30

/data/html/2023/07:
19

/data/html/2023/07/19:
2023-07-19-65.bin  index.html

/data/html/2023/week-30:
index.html
```

The main `index.html` file is the entry point for the whole incremental report. Then there is a tree that has a directory for the year (2023), the month (07), and the day (19), and an `index.html` file for that day.

There is also a part of the tree that collects data for the current week; in this case, week number 30. The tree is therefore going to be expanded as more days come into play. The special `LAST_PARSED` file is used by pgBadger to remember when it stopped parsing, allowing it to start from there at the very next incremental invocation.

If you point your web browser to the main index file, you will see a calendar like the one in the following screenshot, where you can select the month and day to see the per-day report.

Year 2023						
July						
	Su	Mo	Tu	We	Th	Fr
27						
28	02	03	04	05	06	07
29	09	10	11	12	13	14
30	16	17	18	19	20	21
31	23	24	25	26	27	28
32	30	31				

Figure 14.6: pgBadger global dashboard over a range of days

Clicking on a specific day, you will be redirected to the daily report, which shows the exact same dashboards already discussed. Clearly, you cannot click days for which the report has not been generated yet or that do not have corresponding activity in the PostgreSQL logs.

Thanks to the incremental approach, you can now schedule the execution in your own scheduler; for example, in `cron(1)` you can insert a line like the following:

```
59 23 * * * pgbadger -I --outdir /data/html/ -f stderr /postgres/16/data/
log/postgresql- 'date +%Y-%m-%d' '.log
```

That is essentially the same command line as the preceding one, with the current date automatically computed. The preceding line will produce, at the end of every day, the report for the current day, using it to populate your report tree.



The previous crontab entry is just an example. Please consider wrapping everything in a robust script and testing the correctness of its execution.

Lastly, it is possible to run pgBadger from a remote host, so that you can dedicate a single machine to collecting all the reports and information in a single place. In fact, pgBadger accepts a URI parameter that is the remote location of the log directory (or file) and can be accessed via either FTP or the more secure and recommended SSH.

As an example, the following represents the same command line as seen previously, which pulls, in incremental mode, the logs from a remote PostgreSQL host named miguel:

```
$ pgbadger -I --outdir /data/html ssh://postgres@miguel//postgres/16/
data/log/postgresql-'date +%Y-%m-%d'.pgbadger.log
[=====>] Parsed 313252 bytes of 313252 (100.00%),
queries: 841, events: 34
```

Please note that the log file has been specified via an SSH URL. It is highly recommended to use a remote user that has access to the logs and perform an SSH key exchange to automate the login between the hosts.

Now that we know how to use logs, we will move on to another way of looking at tasks—auditing.

Implementing auditing

Auditing is the capability of performing introspection over an application or user session, in other words, to be able to reproduce, step by step, what the user or the application asked the cluster to do.

Auditing is slightly different from logging, as logging provides a simple way of saving actions of the user, but without providing an easy way to reconstruct the user or application interactions with the cluster. In fact, in a highly concurrent cluster, many actions made by different users will coexist in the logs in a mixed bunch of lines. Moreover, logging does not provide any particular logic on what it is storing, and therefore it becomes hard to find out what a user has done. This becomes even more true when the user or the application executes complex statements, in particular, statements where parameters and values are not explicitly provided.

As an example, consider the following simple section:

```
forumdb=> PREPARE my_query( text ) AS SELECT * FROM forum.categories WHERE
title like $1;
PREPARE
forumdb=> EXECUTE my_query( 'PROGRAMMING%' );
pk |          title          |          description
----+-----+-----
```

```
3 | PROGRAMMING LANGUAGES | All about programming languages
(1 row)
```

That will reveal, with verbose logging, the following:

```
LOG:  duration: 19.011 ms  statement: PREPARE my_query( text ) AS SELECT *
FROM forum.categories WHERE title like $1;
LOG:  duration: 6.539 ms  statement: EXECUTE my_query( 'PROGRAMMING%' );
```

As you can see, in the logs, there is everything you need to reconstruct what the user has done, but that is not so simple. You have to understand that the two lines are related to each other and that the session from which the statements have been executed is the same. This is not always possible—especially if other queries are logged between the two lines you are interested in.

Moreover, it could happen that the logs do not report all the information you need—perhaps because you chose to not log statements that execute faster than a threshold.

Therefore, while you can use logging to perform auditing, that is not always the best choice. In this section, you will learn about the PgAudit extension, which was created to provide a reliable and easy-to-use auditing infrastructure. PgAudit exploits the excellent PostgreSQL logging facility; therefore, you need to configure your logging infrastructure in an appropriate way, as you will see in the next subsections.

Before we dig into the configuration and usage of PgAudit, there are some details and concepts that have to be explained. PgAudit can work in two different ways: auditing by session or by object. The former is a quick and simple way to audit a part of (or a whole) session by a user or an application; the latter is a more complex and fine-grained way of logging actions related to specific database objects (for example, who deleted rows from that table?).

Auditing by session works by simply configuring the categories of statements to audit within a session. On the other hand, auditing by object requires you to configure individual database roles that, depending on their set of permissions, will trigger the auditing of specific actions. In the following subsections, you will see both ways used to audit.

Installing PgAudit

The fastest way to install PgAudit is to use the operating system package manager. For example, on a GNU/Debian or Ubuntu system you can type:

```
$ sudo apt install pgaudit-16-pgaudit
```

If you need to install PgAudit from the source, you first need to grab a version compatible with your own PostgreSQL cluster, then uncompress and install it:

```
$ wget https://github.com/pgaudit/pgaudit/archive/refs/tags/1.7.0.tar.gz
..
$ tar xzvf 1.7.0.tar.gz
...
$ cd pgaudit-1.7.0
$ make USE_PGXS=1
...

$ sudo make USE_PGXS=1 install
```

Once the extension is installed, you have to configure PostgreSQL to use PgAudit.

Configuring PostgreSQL to exploit PgAudit

PgAudit is an extension that needs to be loaded at server startup, therefore you have to change the main configuration file, `postgresql.conf`, to include the `pgaudit` library as follows:

```
shared_preload_libraries = 'pgaudit'
```



If you are using the Docker image for this chapter, PgAudit is already installed for you!

Then, restart your cluster to make the changes take effect:

```
$ pg_ctl -D /postgres/16/data restart
```

Since PgAudit is an extension, you have to enable it within the database you want to audit in order to activate it. For the sake of simplicity, let's enable it within our `forumdb` database (you need to connect as a database superuser):

```
forumdb=# CREATE EXTENSION pgaudit;
CREATE EXTENSION
```

It is now time to decide when and how to apply auditing.

Configuring PgAudit

PgAudit ships with a rich set of configuration parameters that allow you to specify exactly what to log, when, what to exclude from auditing, and so on. All configuration parameters live within the `pgaudit` namespace so that they will not clash with other existing settings with the same name.

The most important setting is `pgaudit.log`, which defines which statements and actions you want to audit. The parameter can assume any of the following values:

- `ALL` to audit every statement
- `NONE` to audit nothing at all
- `READ` to audit only `SELECT` and `COPY` statements
- `WRITE` to audit every statement that modifies data (`INSERT`, `UPDATE`, and `COPY`)
- `ROLE` to audit role changes or creation
- `DDL` to audit all the data-definition statements, and therefore any change to the database structure
- `FUNCTION` to audit all code execution, including `DO` blocks
- `MISC` to audit all the values not explicitly categorized above
- `MISC_SET` to audit all `SET`-like commands

You are free to specify more than one setting at the same time by separating single names with a comma, for example:

```
pgaudit.log = 'WRITE,FUNCTION';
```

This function can be used to audit all data changes and code executions.

Another important configuration parameter is `pgaudit.log_level`, which specifies the log level that PgAudit will use to make the auditing messages appear in the logs. By default, this setting assumes the value `log`, but you can change it to any other log threshold except error ones (such as `ERROR`, `FATAL`, and `PANIC`).

In order to insert more details in the audit information, you will likely want to enable `pgaudit.log_parameter` to dump any query parameters (you will see an example later).

If you are going to configure PgAudit by object, you need to set the `pgaudit.role` parameter as you will see later in this chapter.